Kernel Normalization for Vision Transformers and Dense Convolutional Networks

Conor Brennan
conorb23@student.ubc.ca

Jordan Bourak
jbourak@pm.me

Abstract

Most vision learning architectures today rely on BatchNorm or LayerNorm normalization. However, the recently proposed KernelNorm was shown to result in higher or equal performance when used in ResNets over BatchNorm, which may be the first instance where a batch-independent normalization method has outperformed BatchNorm at high batch sizes. As KernelNorm can be considered a special case of LayerNorm, it could also be applied to any architecture that uses LayerNorm. Consequently, we look to further investigate the performance of KernelNorm within the CNN architecture and experiment with KernelNorm in ViTs.

1 Introduction

The standard normalization techniques used in various architectures has remained mostly the same for the last several years. In the vision learning space, vision transformers (ViTs)(Dosovitskiy et al. 2021) most commonly use LayerNorm (Ba, Kiros, and Hinton 2016) while convolutional neural networks (CNNs) rely primarily on BatchNorm (Ioffe and Szegedy 2015). However, Nasirigerdeh, R. Torkzadehmahani, et al. (2024) recently proposed KernelNorm, an alternative batch-independent normalization developed to overcome limitations of existing techniques in both private and non-private learning settings.

It was demonstrated that a KernelNorm variant of deep residual networks (ResNets) (He et al. 2016) had higher or about equal performance and faster convergence (measured by number of iterations) for semantic segmentation and image classification tasks, compared to several other normalization methods. These benefits come at the cost of longer training and test times, as well as higher memory usage. However, as computing power continues to increase over time, it is important to explore the usage of KernelNorm in other model architectures since KernelNorm may be the first normalization method that has has at least equal performance of BatchNorm while keeping all samples in a batch separate.

As ViTs and CNNs are both still relevant in the field of vision learning today, we look to expand the understanding of the benefits of KernelNorm within both of these architectures. In particular, firstly we aim to evaluate KernelNorm equivalents of a subset of ViTs, which use primarily use LayerNorm. Although ViTs are vastly different from CNNs, because LayerNorm can be viewed as a special case of KernelNorm, KernelNorm can indeed be applied to transformers. Secondly, we evaluate KernelNorm within a small bottleneck-variant of a Densely Connected Convolutional Network (DenseNet) (Huang, Liu, and Weinberger 2016) architecture. Although DenseNets are not as widely used as other convolution based architectures, they are relevant in a few applications such as medical image analysis (Zhou et al. 2022). As KernelNorm ResNet variants outperformed other normalized counterparts, replacing BatchNorm in DenseNet may also result in performance improvements.

Our contributions are summarized as follows: we find that our small kernel normalized ViTs outperform their typically normalized counterparts, but our small kernel normalized DenseNets performed worse than their BatchNorm counterparts.

Course project for UBC CPSC 440/550, April 2024.

2 Related Work

2.1 Normalization Methods

In general, all normalization methods fall into two categories: batch-dependant and batch-independent. Batch-dependant methods have the batch size built into the normalization method. Normalization methods that fall into this category include BatchNorm as well as its derivatives, including batch renormalization (Ioffe 2017) and cross-iteration BatchNorm (Yao, Cao, Zheng, et al. 2020) among others. Batch-Independent methods include LayerNorm (Ba, Kiros, and Hinton 2016), GroupNorm (Wu and He 2018), and InstanceNorm (Ulyanov, Vedaldi, and Lempitsky 2016) among others.

2.2 Normalization in CNNs

BatchNorm is by far the most dominant method of normalization for CNNs. Although other methods outperform BatchNorm in specific situations, such as GroupNorm with low batch sizes (Wu and He 2018) or InstanceNorm for "stylized" images (Ulyanov, Vedaldi, and Lempitsky 2016), no other method has been shown to have the same generalizability as BatchNorm within the context of visual learning with CNNs. It has been shown that KernelNorm ResNet equivalents can have equal or better performance than the BatchNorm counterparts on image classification and image segmentation (Nasirigerdeh, R. Torkzadehmahani, et al. 2024). However, as mentioned by the authors, further experiments must be conducted in other settings and architectures to gain a better understanding of its performance.

2.3 Normalization in ViTs

As ViTs are a more recent development compared to CNNs, the normalization methods are not as well-established. LayerNorm is most commonly in ViTs, but this is partially due to the fact it was inheritated from original transformers designed for natural language processing, which must be compatible with inputs of varying lengths. However, ViTs will generally have a fixed input size, so they are not restricted to batch-independent normalization methods. ViT variants with BatchNorm have been explored and have been shown to result in better performance as well as higher training and test throughput, measured in images per second (Yao, Cao, Lin, et al. 2021) (Tang and Xie 2023).

3 KernalNorm and KernelNorm Convolution Layers

KernelNorm (Nasirigerdeh, R. Torkzadehmahani, et al. 2024) is an input batch-independent normalization designed to overcome some of the limitations of BatchNorm. Specifically, BatchNorm does not perform well with small batch sizes (Wu and He 2018; Nasirigerdeh, R. Torkzadehmahani, et al. 2024). Further, it cannot be applied in privacy-preserving settings where samples must remain seperate because it is batch-dependant.

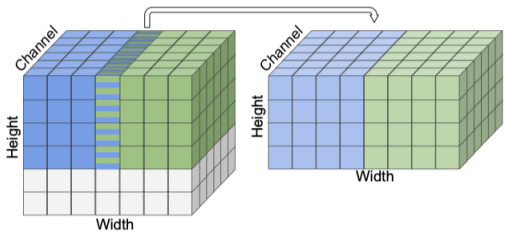

Figure 1: KernelNorm on an input of size $7 \times 7 \times 7$ with kernel size 4, stride 3, and padding 0. Original figure is from Nasirigerdeh, R. Torkzadehmahani, et al. (2024).

KernelNorm is similar to a pooling layer except it normalizes each region specified by the given kernel size, stride, and padding rather than summarizing. Hence, the mean and variance used to normalize each region is calculated by taking into account all channels at once. The key characteristic of KernelNorm is that an input element can contribute multiple times to the output. This means that

it takes into account the spatial correlation between input elements. Or in other words, it takes into account the similarities between an input element and the elements within distance equal to the kernel size. This is not possible with other commonly used normalization methods because they induce a one-to-one correspondence between the input and the output.

A KernelNorm Convolution (KNConv) layer is a KernelNorm layer followed by a 2D Convolution with kernel size and stride equal to that of the kernel size used for KernelNorm. Nasirigerdeh, R. Torkzadehmahani, et al. (2024) have provided implementations for KernelNorm and KernelNorm Convolution (KNConv) via their GitHub repository, which we incorporate into our own KernalNorm based architectures.

4 Kernal Normalized ViTs

First, we outline the the structure of ViTs in order to identify the location of the normalization methods within the architecture. As transformers were built for sequential inputs, ViTs first split an input image into "patches" (Vaswani et al. 2017), akin to cutting an image along a grid with scissors, and then flattens it. This image "sequence" is fed through several layers of encoder blocks. Each encoder block is composed of a multi-head attention (MHA) layer and a multilayer perceptron (MLP), each paired with a normalization layer. The most commonly used normalization layer in these blocks is LayerNorm.

Initial ViTs were often very sensitive during optimization. However, Xiao et al. (2021) demonstrated that ViTs with a small number of initial convolutions with BatchNorm lead to better optimization stability and and increased performance. This finding was adopted as a standard part of many ViT designs, including the ViT models in the PyTorch library.

In order to implement kernel normalized ViTs (KNViTs), we adapt the PyTorch implementation of ViTs (PyTorch 2023) and proceed by first replacing the BatchNorm normalization layer that is applied before the transformer portion of the architecture. This requires only replacing the BatchNorm layer and convolution layer with a single KNConv layer from PyTorch (2023).

We next look to modify the standard EncoderBlock. An EncoderBlock takes in an input and performs two normalizations within the block. The input has a batch size of n with s e-dimensional vectors, giving a 3D tensor of shape (n, s, e), where e is the `hidden_dim` of the ViT.

However, KernelNorm requires a 4D input tensor of shape (n, c, h, w) so we cannot use it directly. To solve this, we implemented FlatKernelNorm as a modification of the KernelNorm layer that can handle 3D tensors. In our implementation of FlatKernelNorm we assume that e is a square number so we can reshape the input tensor to $(n, s, \sqrt{e}, \sqrt{e})$. This requires us to restrict our KNViT models to square `hidden_dim`.

Recall the design of the original EncoderBlock. If we have an input i, then we have

$$x = i + MHA(LayerNorm(i)),$$

followed by

$$y = x + MLP(LayerNorm(x))$$

where the output is y. Because the unnormalized i and x are added to their normalized counterparts following the MHA and MLP layers respectively, the shape of the output from FlatKernelNorm must remain the same as the input. Hence, we set the kernel size equal to the stride length. In doing this, note that we induce a one-to-one correspondence between the input and output of the normalization and lose some of the unique spatial correlation properties of KernelNorm.

With these modifications to the ViT architecture, we have a kernel normalized KNViT model as outlined in Figure 2. We put forward two KNViT models, named by a similar scheme to those of PyTorch. The KNViT-S-16 is similar to PyTorch's ViT-B-16 but has 14 heads instead of 12 for the MHA and `hidden_dim` of 784 (28^2) instead of 768. The KNViT-S-8 is the same as the KNViT-S-16 but has patch size 8 instead of 16.

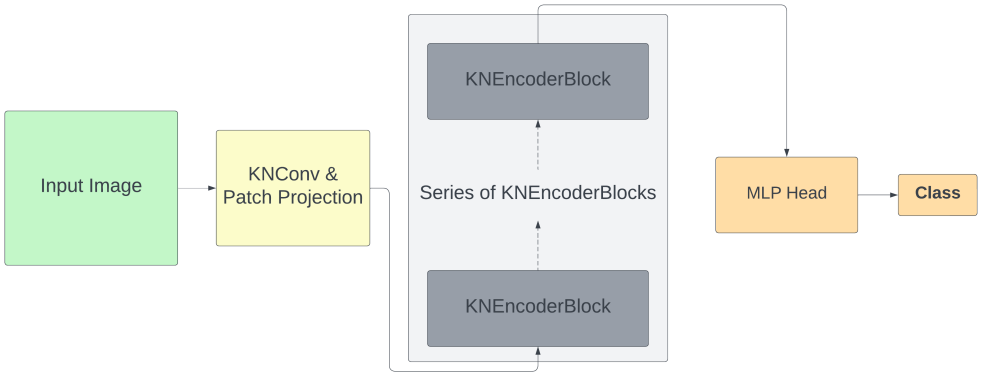

Figure 2: KNViT architecture

5 Kernel Normalized DenseNets

As before, we briefly outline the structure of DenseNets to understand how we can implement KernelNowm into the architecture. The original DenseNets essentially uses a sequence of 3×3 and 1×1 convolutions, each paired with a normalization layer (except the first convolution) and activation layer, followed by the final classification layer. Although DenseNets have several variants, here we only consider the bottleneck variant, as it is much more parameter efficient (Huang, Liu, and Weinberger 2016). The aforementioned convolutions are organized into dense blocks, which have the bottleneck built as a 1×1 convolution before the typical 3×3 convolution, and transition layers, which use a single 1×1 convolution. The resulting structure is somewhat similar to ResNets, but each dense block produces an output that is concatenated to the input rather than summed.

To implement a kernel normalized DenseNet (KNDenseNet), we adapt the implementation of DenseNets from (Veit 2017). We proceed by replacing all of the BatchNorm layers by their KernelNorm equivalent. The adapted implementation has the activation layer in between the normalization and convolution layers, so we re-order the layers in order to use the KNConv layer implementations from (Nasirigerdeh, R. Torkzadehmahani, et al. 2024). By replacing the BatchNorm + convolution layer pairs with a single KNConv layer, the resulting architecture is what we name a KNDenseNet.

6 Experiments

6.1 KNViTs

We look to compare our KNViTs with their closest typical ViT counterparts. PyTorch does not have built-in models that are comparable to the size of KNViT-S-8 and KNViT-S-16 with 117 million parameters, with the closest being ViT-B-16 with 87M and ViT-L-16 with 304M. So, we constructed our own two equivalent models ViT-S-8 and ViT-S-16 that have an identical number of parameters to their KernelNorm counterparts. We compared the KNViTs and ViTs on the CIFAR10 dataset.

Data preparation: the data were pre-processed using the same procedure as in (Nasirigerdeh, R. Torkzadehmahani, et al. 2024), where the samples are padded (size 4×4) and then flipped and cropped (with size 32×32).

Training: the models were trained for 100 epochs on batches of size 64, using stochastic gradient descent with momentum 0.9 and an initial learning rate of 0.025. Learning rates were scheduled using cosine annealing.

Results: for both KNViTs and ViTs, we see that patch sizes of 8 outperform patch sizes of 16, as was previously shown by Beyer et al. (2023). It is also apparent that the KNViT models converge faster and achieve higher test accuracy than their counterparts, similar to the findings of Nasirigerdeh, R. Torkzadehmahani, et al. (2024) for KNResNets.

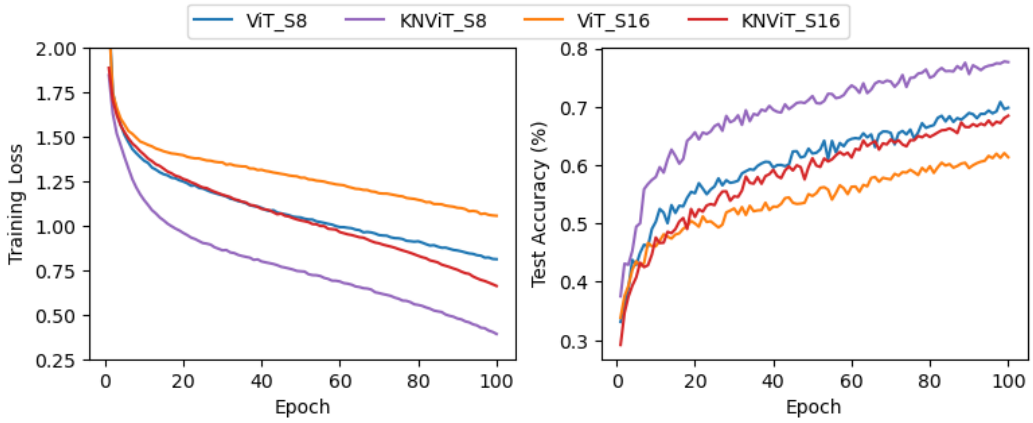

Figure 3: Training loss and test accuracy for 4 small KN/ViTs, with patches of size 8 and 16

6.2 KNDenseNets

We trained a single bottlenecked and compressed DenseNet (DenseNet-BC) variant for each of BatchNorm (as in the original implementation), KernelNorm, as well as a mixed norm variation. The mixed norm variation utilizes KernelNorm for the 3×3 convolution and BatchNorm for the 1×1 convolution, and was developed in attempt to gain a better understanding of the poor performance of KNDenseNet details below. Due to resource availability, we used a fairly small variation of DenseNets with depth $L = 40$ and growth rate of $k = 12$.

Data preparation: the data were pre-processed using the same procedure as in (Nasirigerdeh, R. Torkzadehmahani, et al. 2024), where the samples are padded (size 4×4) and then flipped and cropped (with size 32×32).

Training: we retain the training procedure as used by Huang, Liu, and Weinberger (2016). We train the models for 300 epochs on batches of size 64 with weight decay of 1×10^{-4}, using stochastic gradient descent with momentum 0.9 and an initial learning rate of 0.1. Further, at epochs 150 and 225, we set the learning rate manually to 10% of the initial learning rate.

Results: we see that replacing any BatchNorm normalization layer with KernelNorm resulted in lower test accuracy. Furthermore, the test accuracy decreases after adjusting the learning rate at epoch 150 for the BatchNorm and mixed norm variants, but this is similar to the behaviour of the test accuracies in (Huang, Liu, and Weinberger 2016). The KernelNorm variants do however still appear to retain the property of faster convergence observed in KNResNets (Nasirigerdeh, R. Torkzadehmahani, et al. 2024).

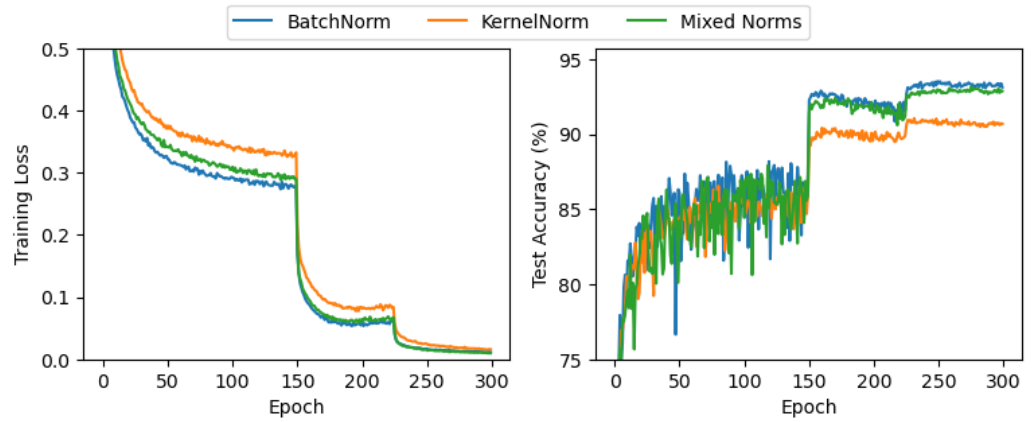

Figure 4: Training loss and test accuracy for BatchNorm, KernelNorm, and mixed norm variants of DenseNet-BC with $k = 12$ and $L = 40$.

7 Discussion

KNViTs showed a significant increase in accuracy compared to the non-kernel normalized variants. Recall the restrictions of equal kernel size and stride for KernelNorm that we imposed when implementing KNViTs, which impaired its ability to incorporate spatial correlation. By further modifying the KNViT architecture in a way that would allow unequal kernel size and stride for KernelNorm, we could potentially see further improvements. Due to the time constraints of the project and limited hardware, we did very little hyper-parameter optimization for KNViTs and ViTs. For the same reasons, we did not test larger scale KNViTs, which could have as many as 632M parameters as in ViT-Huge from (Dosovitskiy et al. 2021) Finally, our results point to potential gains in models based on the ViT architecture, such as Conformers (Gulati et al. 2020).

Contrarily, KNDenseNets performed worse than their BatchNorm counterparts. A potential explanation for this is the bottleneck layers as well as the concatenation aspect of DenseNet. By design, these result in many more changes in the dimensionality of the channels in the input many times as it feeds through the model, and the meaning of spatial correlation becomes slightly more "convoluted." This partially aligns with the findings in (Nasirigerdeh, R. Torkzadehmahani, et al. 2024), where larger KNResNets with bottleneck layers performed about the same as smaller variants with no bottleneck layers. On the other hand, our findings partially conflict with those of (Nasirigerdeh, J. Torkzadehmahani, et al. 2022), who found that adding KernelNorm to DenseNet in a privacy-preserving setting resulted in higher test accuracy. However, their KNDenseNet implementation used a 3×3 KNConv layer inside the transition layer rather than a 1×1 convolution as in the original DenseNet, which would result in a higher number of parameters and thus may be the underlying reason for the increase in performance.

In conclusion, we suggest a more thorough analysis of the KernelNorm models presented here, including hyperparameter tuning and method of implementation, especially for KNViTs which do not currently utilize KernelNorm to its full extent in the forms presented here. Future work could also involve experimenting with higher resolution datasets where KernelNorm's ability to incorporate spatial correlation may be more impactful. Lastly, as we were highly limited by computational resources, scaling up the models could also provide a more accurate understanding of their potential.

References

Ba, Jimmy Lei, Jamie Ryan Kiros, and Geoffrey E Hinton (2016). "Layer normalization." *arXiv preprint arXiv:1607.06450*.

Beyer, Lucas, Pavel Izmailov, Alexander Kolesnikov, Mathilde Caron, Simon Kornblith, Xiaohua Zhai, Matthias Minderer, Michael Tschannen, Ibrahim Alabdulmohsin, and Filip Pavetic (2023). *FlexiViT: One Model for All Patch Sizes*. eprint: 2212.08013.

Dosovitskiy, Alexey, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby (2021). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. arXiv: 2010.11929 [cs.CV].

Gulati, Anmol, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, and Ruoming Pang (2020). *Conformer: Convolution-augmented Transformer for Speech Recognition*. arXiv: 2005.08100 [eess.AS].

He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun (2016). "Deep residual learning for image recognition." *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778.

Huang, Gao, Zhuang Liu, and Kilian Q. Weinberger (2016). "Densely Connected Convolutional Networks." *CoRR* abs/1608.06993.

Ioffe, Sergey (2017). "Batch Renormalization: Towards Reducing Minibatch Dependence in Batch-Normalized Models." *Advances in Neural Information Processing Systems*. Ed. by I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett. Vol. 30. Curran Associates, Inc.

Ioffe, Sergey and Christian Szegedy (2015). "Batch normalization: Accelerating deep network training by reducing internal covariate shift." *International conference on machine learning*. PMLR, pp. 448–456.

Nasirigerdeh, Reza, Javad Torkzadehmahani, Daniel Rueckert, and Georgios Kaissis (2022). *Kernel Normalized Convolutional Networks for Privacy-Preserving Machine Learning*. eprint: 2210.00053.

Nasirigerdeh, Reza, Reihaneh Torkzadehmahani, Daniel Rueckert, and Georgios Kaissis (2024). "Kernel Normalized Convolutional Networks." *Transactions on Machine Learning Research*. ISSN: 2835-8856.

PyTorch (2023). *VisionTransformer PyTorch implementation*. https://pytorch.org/vision/main/models/vision_transformer.html.

Tang, Xue-Song and Xian-Lin Xie (2023). "Re-Introducing BN Into Transformers for Vision Tasks." *IEEE Access* 11, pp. 58462–58469. DOI: 10.1109/ACCESS.2023.3283612.

Ulyanov, Dmitry, Andrea Vedaldi, and Victor S. Lempitsky (2016). "Instance Normalization: The Missing Ingredient for Fast Stylization." *CoRR* abs/1607.08022.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin (2017). "Attention is All you Need." *Advances in Neural Information Processing Systems*.

Veit, Andreas (2017). *A PyTorch Implementation for Densely Connected Convolutional Networks (DenseNets)*. https://github.com/andreasveit/densenet-pytorch?tab=readme-ov-file.

Wu, Yuxin and Kaiming He (Sept. 2018). "Group Normalization." *Proceedings of the European Conference on Computer Vision (ECCV)*.

Xiao, Tete, Mannat Singh, Eric Mintun, Trevor Darrell, Piotr Dollár, and Ross B. Girshick (2021). "Early Convolutions Help Transformers See Better." *CoRR* abs/2106.14881. arXiv: 2106.14881. URL: https://arxiv.org/abs/2106.14881.

Yao, Zhuliang, Yue Cao, Yutong Lin, Ze Liu, Zheng Zhang, and Han Hu (2021). "Leveraging Batch Normalization for Vision Transformers." *2021 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, pp. 413–422. DOI: 10.1109/ICCVW54120.2021.00050.

Yao, Zhuliang, Yue Cao, Shuxin Zheng, Gao Huang, and Stephen Lin (2020). "Cross-Iteration Batch Normalization." *CoRR* abs/2002.05712.

Zhou, T., X. Ye, H. Lu, X. Zheng, S. Qiu, and Y. Liu (Apr. 25, 2022). "Dense Convolutional Network and Its Application in Medical Image Analysis. Biomed Res Int." *2022:2384830. ; PMCID:* PMC9060995. PMID: 35509707. DOI: 10.1155/2022/2384830.